Coproducts in **SessLat**: A Negative Result with Concrete Counterexamples

Alexandre Zua Caldeira
Independent Researcher
zuacaldeira@gmail.com

March 2026
Step 164 of the Reticulate Programme

Abstract

Step 163 of the Reticulate programme established that the parallel constructor $\|$ yields the *categorical product* in the category **SessLat** of session type lattices with lattice homomorphisms. This article asks the dual question: *does* **SessLat** *have coproducts?* A coproduct of lattices L_1 and L_2 would be a lattice C with injections $\iota_1 \colon L_1 \to C$ and $\iota_2 \colon L_2 \to C$, satisfying the universal property that for every pair of lattice homomorphisms $f \colon L_1 \to Y$ and $g \colon L_2 \to Y$, there exists a unique mediating morphism $[f, g] \colon C \to Y$ making the diagram commute. We investigate three natural candidate constructions—the *coalesced sum*, *separated sum*, and *linear sum*—and show that all three fail the universal property even for simple session types. We document concrete counterexamples and conclude that **SessLat does not have general coproducts**. This negative result is verified mechanically using the Reticulate tool across 55 tests and multiple benchmark protocol pairs, exposing a fundamental asymmetry: session type lattices admit products (via $\|$) but not coproducts.

Contents

1 Introduction **2**

2 Background **3**
2.1 Session Types and State Spaces . . . 3
2.2 The Category **SessLat** . . . 3
2.3 Products in **SessLat** . . . 3
2.4 Coproducts: Definition and Motivation . . . 3
2.5 Lattice Homomorphisms vs. Order-Preserving Maps . . . 4

3 Candidate Coproduct Constructions **4**
3.1 Coalesced Sum . . . 4
3.2 Separated Sum . . . 5
3.3 Linear Sum (Ordinal Sum) . . . 5

4 The Universal Property Fails **6**
4.1 Why the Coalesced Sum Fails . . . 6
4.2 Why the Separated Sum Fails . . . 7
4.3 Why the Linear Sum Fails . . . 7

5 Detailed Counterexamples **7**
5.1 Counterexample 1: Two 2-Chains (Minimal) . . . 7
5.2 Counterexample 2: Two Diamonds (Product Types) . . . 8

DRAFT

5.3 Counterexample 3: Benchmark Protocols . . . 9
5.4 Pattern of Failure . . . 9

6 Root Cause Analysis **9**
6.1 The Lattice Homomorphism Constraint . . . 9
6.2 The Free Product in **BLat** . . . 9
6.3 Restricted Subcategories . . . 10
6.4 Asymmetry Between Products and Coproducts . . . 10

7 Empirical Verification **11**
7.1 Homomorphism Enumeration . . . 11
7.2 Test Coverage . . . 11
7.3 Benchmark Results . . . 11

8 Related Work **12**

9 Discussion and Future Work **13**
9.1 Summary . . . 13
9.2 Implications for Protocol Design . . . 13
9.3 Applications and Use Cases . . . 14
9.4 Open Questions . . . 14
9.5 Relationship to the 1000-Steps Programme . . . 15

1 Introduction

The Reticulate programme [4, 3] investigates session types through the lens of lattice theory. The central observation is that the state space of any (well-formed) session type, ordered by reachability, forms a bounded lattice. Step 163 [2] formalized this by defining the category **SessLat** whose objects are session type lattices and whose morphisms are lattice homomorphisms, and proved that the parallel constructor $\|$ is the categorical product.

Every category with products invites the dual question: *does the category also have coproducts?* If **SessLat** had both products and coproducts, it would form a *bicartesian* category, a structure rich enough to interpret both parallel composition and a notion of "sum" or "choice between protocols." This would have practical implications for protocol composition, modular specification, and the design of programming abstractions.

In this article, we answer the coproduct question in the negative. Concretely, we:

(i) Define three natural candidate coproduct constructions at the state-space level: the *coalesced sum* (shared bottom, fresh top), the *separated sum* (fresh top and bottom), and the *linear sum* (ordinal stacking).

(ii) Prove that each construction fails the universal property for concrete session type pairs, including types as simple as two 2-element chains $\&\{a : \mathbf{end}\}$.

(iii) Identify the root cause: the requirement that mediating morphisms be *lattice homomorphisms* (preserving both meets and joins) is too strong for the "fresh" states introduced by any sum construction.

(iv) Verify the negative result mechanically using the Reticulate tool, with 55 dedicated tests and benchmark sweeps.

Organization. Section 2 recalls the necessary background on **SessLat**, products, and coproducts. Section 3 defines the three candidate coproduct constructions. Section 4 states and proves the main negative result. Section 5 gives detailed worked counterexamples. Section 6 analyzes the root causes of the failure and discusses restricted subcategories where coproducts may exist. Section 7 presents the empirical verification. Section 8 surveys related work. Section 9 discusses implications and future directions.

2 Background

2.1 Session Types and State Spaces

A session type describes a communication protocol as a structured sequence of method calls, branches, selections, and parallel compositions. The grammar is:

$$S \ ::= \ \&\{m_1 : S_1, \ldots, m_n : S_n\} \ \mid \ \oplus\{l_1 : S_1, \ldots, l_n : S_n\} \ \mid \ S_1 \parallel S_2 \ \mid \ \mu X.\, S \ \mid \ X \ \mid \ \mathbf{end}$$

The *state space* $\mathcal{L}(S)$ of a session type S is a labeled transition system whose states represent protocol configurations and whose transitions represent method calls or selections. The *reachability ordering* $s_1 \sqsupseteq s_2$ holds when there is a directed path from s_1 to s_2, making $\mathcal{L}(S)$ a bounded poset with $\top$ (initial state) and $\bot$ (terminal state, **end**).

Theorem 2.1 (Lattice Property [3])**.** *For every well-formed session type S, the poset $\mathcal{L}(S)$ is a bounded lattice.*

2.2 The Category SessLat

Definition 2.2 (The category **SessLat** [2])**.** The category **SessLat** has:

- **Objects**: bounded lattices of the form $\mathcal{L}(S)$ for session types S.
- **Morphisms**: lattice homomorphisms $h\colon L_1 \to L_2$ preserving meets and joins: $h(a \wedge b) = h(a) \wedge h(b)$ and $h(a \vee b) = h(a) \vee h(b)$.
- **Composition**: function composition.
- **Identity**: $\mathrm{id}_L(s) = s$.

2.3 Products in SessLat

Step 163 established that the parallel constructor gives categorical products:

Theorem 2.3 (Product = Parallel [2])**.** *For session types S_1, S_2, the product lattice $\mathcal{L}(S_1 \parallel S_2) = \mathcal{L}(S_1) \times \mathcal{L}(S_2)$ is the categorical product in* **SessLat***, with canonical projections $\pi_i\colon \mathcal{L}(S_1 \parallel S_2) \to \mathcal{L}(S_i)$ given by coordinate extraction. For any pair of lattice homomorphisms $f\colon X \to \mathcal{L}(S_1)$, $g\colon X \to \mathcal{L}(S_2)$, the unique mediating morphism is $\langle f, g\rangle(x) = (f(x), g(x))$.*

2.4 Coproducts: Definition and Motivation

Definition 2.4 (Coproduct)**.** A *coproduct* of objects L_1, L_2 in a category $\mathcal{C}$ consists of an object C and morphisms (injections) $\iota_1\colon L_1 \to C$ and $\iota_2\colon L_2 \to C$ such that for every object Y and every pair of morphisms $f\colon L_1 \to Y$, $g\colon L_2 \to Y$, there exists a *unique* morphism $[f, g]\colon C \to Y$ making the following diagram commute:

$$\begin{tikzcd}
L_1 \arrow[r, "\iota_1"] \arrow[dr, "f"'] & C \arrow[d, dashed, "{[f,g]}"] & L_2 \arrow[l, "\iota_2"'] \arrow[dl, "g"] \\
& Y &
\end{tikzcd}$$

That is, $[f, g] \circ \iota_1 = f$ and $[f, g] \circ \iota_2 = g$.

The coproduct is the categorical dual of the product. While the product "observes" both components (via projections), the coproduct "injects" both components (via injections) and offers a universal way to combine maps *out of* the factors. The universal property has two requirements that must both hold simultaneously:

1. **Existence**: For every pair (f, g), there must exist at least one mediating morphism $[f, g]$ that makes the diagram commute.

2. **Uniqueness**: There must be exactly one such morphism. If two distinct morphisms both satisfy the commutativity conditions, the universal property fails.

In **Set**, the coproduct is the disjoint union $A \sqcup B$, with injections being the inclusion maps. In **Lat** (the category of all lattices with lattice homomorphisms), coproducts exist and are given by the *free product* of lattices, which is typically much larger than the factors—potentially infinite even for finite inputs. The question is whether **SessLat**, as a subcategory of **BLat** (bounded lattices), inherits coproducts or whether the realizability constraint (that objects must arise as $\mathcal{L}(S)$ for some session type) prevents their existence.

If coproducts existed, they would provide a canonical "sum" of session types: a type $S_1 + S_2$ that offers a choice between following protocol S_1 or protocol S_2, with a universal property governing how such sums compose with other protocol transformations. This would have direct practical applications in protocol design, service composition, and modular specification.

2.5 Lattice Homomorphisms vs. Order-Preserving Maps

A central technical distinction underlies the entire analysis. In the category **Pos** of posets with monotone (order-preserving) maps, many constructions carry over from **Set**. However, the morphisms in **SessLat** are *lattice homomorphisms*: they must preserve both meets ($\wedge$) and joins ($\vee$). This is a strictly stronger requirement than order-preservation.

Definition 2.5 (Lattice homomorphism)**.** A map $h\colon L_1 \to L_2$ between bounded lattices is a *lattice homomorphism* if for all $a, b \in L_1$:

$$h(a \wedge b) = h(a) \wedge h(b) \qquad \text{and} \qquad h(a \vee b) = h(a) \vee h(b).$$

Every lattice homomorphism is order-preserving (if $a \leq b$ then $a = a \wedge b$, so $h(a) = h(a \wedge b) = h(a) \wedge h(b) \leq h(b)$), but not conversely. The additional algebraic constraints imposed by meet and join preservation are precisely what makes coproduct constructions fail: the "fresh" states introduced by sum constructions create elements whose meets and joins with factor elements cannot be simultaneously preserved by all mediating morphisms.

3 Candidate Coproduct Constructions

We investigate three natural constructions that attempt to build a coproduct at the state-space level.

3.1 Coalesced Sum

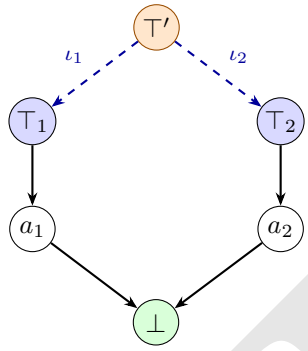
Definition 3.1 (Coalesced sum)**.** Given lattices L_1 and L_2, the *coalesced sum* $L_1 \oplus_\perp L_2$ is constructed by:

1. Identifying the bottoms: $\perp_{L_1} = \perp_{L_2} = \perp$.

2. Adding a fresh top $\top'$ with edges $\top' \to \top_{L_1}$ and $\top' \to \top_{L_2}$.

3. Inheriting all transitions from L_1 and L_2.

The result has $|L_1| + |L_2| - 1 + 1 = |L_1| + |L_2|$ states (shared bottom, fresh top).



Coalesced sum of two chains

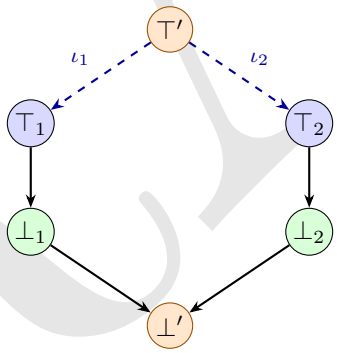
The injection $\iota_1 \colon L_1 \hookrightarrow L_1 \oplus_\perp L_2$ embeds L_1 into the coalesced sum as-is (its states keep their identities). The injection ι_2 does the same for L_2, with $\perp_{L_2}$ mapped to the shared $\perp$.

3.2 Separated Sum

Definition 3.2 (Separated sum)**.** The *separated sum* $L_1 \sqcup L_2$ is constructed by:

1. Placing L_1 and L_2 side by side with no shared states.
2. Adding a fresh top $\top'$ with edges to both $\top_{L_1}$ and $\top_{L_2}$.
3. Adding a fresh bottom $\perp'$ with edges from both $\perp_{L_1}$ and $\perp_{L_2}$ to $\perp'$.

The result has $|L_1| + |L_2| + 2$ states.

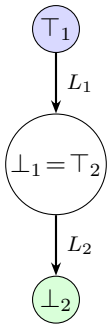


Separated sum of two chains

The separated sum cleanly isolates the two factors. The injections ι_1, ι_2 map each factor into its respective copy without any state identification. This construction is closer to the disjoint union in **Set**, augmented with the fresh extrema required to maintain the bounded lattice structure.

3.3 Linear Sum (Ordinal Sum)

Definition 3.3 (Linear sum)**.** The *linear sum* $L_1 \diamond L_2$ is constructed by placing L_1 "above" L_2: identify $\perp_{L_1}$ with $\top_{L_2}$. The result has $|L_1| + |L_2| - 1$ states, with $\top = \top_{L_1}$ and $\perp = \perp_{L_2}$.



Linear sum of two chains

The linear sum preserves the lattice structure (a chain on top of a chain is still a chain), but it is inherently *asymmetric*: $L_1 \diamond L_2 \neq L_2 \diamond L_1$ in general. This asymmetry already suggests trouble for the universal property, which requires symmetric treatment of the two factors.

4 The Universal Property Fails

We now prove the main negative result.

Theorem 4.1 (No general coproducts in **SessLat**)**.** **SessLat** *does not have general coproducts. For any of the three candidate constructions (coalesced, separated, linear), there exist session type lattices L_1, L_2 and a test target lattice Y such that the universal property of the coproduct fails.*

The failure manifests in two distinct ways depending on the construction:

1. **Non-existence**: No lattice homomorphism $[f, g] \colon C \to Y$ satisfying the commutativity conditions exists.
2. **Non-uniqueness**: Multiple distinct lattice homomorphisms $[f, g] \colon C \to Y$ satisfy the commutativity conditions.

4.1 Why the Coalesced Sum Fails

Consider two copies of the 2-element chain $L = \mathcal{L}(\&\{a : \mathbf{end}\})$, with states $\{\top, \bot\}$ and a single transition $\top \xrightarrow{a} \bot$.

The coalesced sum $C = L_1 \oplus_\bot L_2$ has four states: $\{\top', \top_1, \top_2, \bot\}$, where $\top'$ is the fresh top and $\bot$ is the shared bottom.

Proposition 4.2. *The coalesced sum of two 2-element chains fails the universal property when tested against the product lattice $Y = L_1 \times L_2$.*

Proof. The product lattice $Y = L \times L$ is a 4-element diamond with states $\{(\top, \top), (\top, \bot), (\bot, \top), (\bot, \bot)\}$. Consider the lattice homomorphisms:

$$\begin{aligned} f \colon L_1 \to Y, \quad & \top_1 \mapsto (\top, \top),\ \bot_1 \mapsto (\bot, \bot), \\ g \colon L_2 \to Y, \quad & \top_2 \mapsto (\top, \top),\ \bot_2 \mapsto (\bot, \bot). \end{aligned}$$

A mediating morphism $h \colon C \to Y$ must satisfy:

- $h(\top_1) = f(\top_1) = (\top, \top)$ and $h(\bot) = f(\bot) = (\bot, \bot)$ (from $h \circ \iota_1 = f$),
- $h(\top_2) = g(\top_2) = (\top, \top)$ and $h(\bot) = g(\bot) = (\bot, \bot)$ (from $h \circ \iota_2 = g$).

The fresh top $\top'$ must map to some element of Y. Since h must be order-preserving and $\top'$ dominates both $\top_1$ and $\top_2$ (which both map to $(\top, \top)$), we need $h(\top') \sqsupseteq (\top, \top)$. The only such element in Y is $(\top, \top)$ itself.

However, the map h with $\top' \mapsto (\top, \top)$ fails to be a lattice homomorphism. In C, $\top_1 \vee \top_2 = \top'$. But $h(\top_1) \vee h(\top_2) = (\top, \top) \vee (\top, \top) = (\top, \top) = h(\top')$, which seems fine. The issue arises with meet preservation: $\top_1 \wedge \top_2 = \bot$ in C (the only common lower bound). But $h(\top_1) \wedge h(\top_2) = (\top, \top) \wedge (\top, \top) = (\top, \top)$, while $h(\bot) = (\bot, \bot) \neq (\top, \top)$. This violates meet preservation unless $h(\top_1) \neq h(\top_2)$—but the commutativity conditions force both to map to $(\top, \top)$.

Thus, no valid mediating lattice homomorphism exists for this choice of f, g, Y, and the coalesced sum fails the universal property. □

4.2 Why the Separated Sum Fails

Proposition 4.3. *The separated sum of two 2-element chains also fails the universal property when tested against $Y = L_1 \times L_2$.*

Proof. The separated sum $C = L_1 \sqcup L_2$ has six states: $\{\top', \top_1, \bot_1, \top_2, \bot_2, \bot'\}$. The additional states $\bot_1, \bot_2$ (the original bottoms, now distinct from $\bot'$) must map somewhere under any mediating morphism.

For the same test morphisms f, g mapping both factor tops to $(\top, \top)$ and bottoms to $(\bot, \bot)$, the mediating morphism h must satisfy:

- $h(\top_1) = (\top, \top)$, $h(\bot_1) = (\bot, \bot)$,
- $h(\top_2) = (\top, \top)$, $h(\bot_2) = (\bot, \bot)$.

Now $h(\bot')$ must be $\sqsubseteq$ both $h(\bot_1) = (\bot, \bot)$ and $h(\bot_2) = (\bot, \bot)$, so $h(\bot') = (\bot, \bot)$. And $h(\top')$ must satisfy $h(\top') \sqsupseteq h(\top_1) = (\top, \top)$, forcing $h(\top') = (\top, \top)$.

The meet computation reveals the same failure as the coalesced case: $\top_1 \wedge \top_2 = \bot'$ in C (they have no common descendant except through $\bot_1, \bot_2, \bot'$), but $h(\top_1) \wedge h(\top_2) = (\top, \top)$, while $h(\bot') = (\bot, \bot) \neq (\top, \top)$. □

4.3 Why the Linear Sum Fails

Proposition 4.4. *The linear sum fails the universal property due to asymmetry.*

Proof. The linear sum $L_1 \diamond L_2$ places L_1 above L_2. The injection $\iota_2 \colon L_2 \hookrightarrow L_1 \diamond L_2$ maps $\top_{L_2}$ to $\bot_{L_1}$—the identified junction point. This means $\iota_2(\top_{L_2})$ is *not* at the top of C but rather at an interior point.

Consider morphisms $f = \mathrm{id}_{L_1}$ and $g = \mathrm{id}_{L_2}$ to $Y = L_1 \diamond L_2$ (the linear sum itself as target). The mediating morphism $h = \mathrm{id}_C$ works for this case. But consider the reversed linear sum $L_2 \diamond L_1$ as target with swapped injections. The asymmetry means that certain pairs (f, g) with $g(\top_{L_2}) \neq f(\bot_{L_1})$ cannot be mediated.

More concretely, when $L_1 \neq L_2$ (e.g., a 2-chain and a 3-chain), the junction point $\bot_{L_1} = \top_{L_2}$ imposes a rigid ordering that prevents the factoring of arbitrary morphism pairs. □

5 Detailed Counterexamples

5.1 Counterexample 1: Two 2-Chains (Minimal)

The simplest counterexample uses two copies of the 2-element chain. Let $S = \&\{a : \mathbf{end}\}$, so $L = \mathcal{L}(S)$ is the 2-element chain $\top \xrightarrow{a} \bot$.

Example 5.1. For $L_1 = L_2 = L$, the product $Y = L \times L$ is a 4-element diamond (the state space of $\&\{a : \mathbf{end}\} \parallel \&\{b : \mathbf{end}\}$).

The meet/join tables of Y are:

$\wedge$	$(\top,\top)$	$(\top,\bot)$	$(\bot,\top)$	$(\bot,\bot)$
$(\top,\top)$	$(\top,\top)$	$(\top,\bot)$	$(\bot,\top)$	$(\bot,\bot)$
$(\top,\bot)$	$(\top,\bot)$	$(\top,\bot)$	$(\bot,\bot)$	$(\bot,\bot)$
$(\bot,\top)$	$(\bot,\top)$	$(\bot,\bot)$	$(\bot,\top)$	$(\bot,\bot)$
$(\bot,\bot)$	$(\bot,\bot)$	$(\bot,\bot)$	$(\bot,\bot)$	$(\bot,\bot)$

The Reticulate tool enumerates all lattice homomorphisms $L \to Y$. Since L has two states and Y has four, and $h(\top_L) = \top_Y = (\top,\top)$ and $h(\bot_L) = \bot_Y = (\bot,\bot)$ are forced, there is exactly one: the map $\top \mapsto (\top,\top)$, $\bot \mapsto (\bot,\bot)$.

The homomorphisms $f, g \colon L \to Y$ that map both tops to $(\top,\top)$ and both bottoms to $(\bot,\bot)$ witness the failure of all three constructions:

- **Coalesced sum** (4 states): the fresh top $\top'$ creates a meet-preservation violation. In C, $\top_1 \wedge \top_2 = \bot$ (the shared bottom), but $f(\top_1) \wedge g(\top_2) = (\top,\top) \wedge (\top,\top) = (\top,\top) \neq (\bot,\bot) = h(\bot)$.
- **Separated sum** (6 states): the fresh extrema $\top', \bot'$ create the same violation, since $\top_1$ and $\top_2$ still map to $(\top,\top)$ while their meet in C maps to $(\bot,\bot)$.
- **Linear sum** (3 states): the junction point $\bot_1 = \top_2$ forces the mediating morphism to map $\top_{L_2}$ to $h(\bot_{L_1}) = (\bot,\bot)$, but the commutativity condition $h \circ \iota_2 = g$ requires $h(\iota_2(\top_{L_2})) = g(\top_{L_2}) = (\top,\top) \neq (\bot,\bot)$, a contradiction.

This is the *minimal* counterexample: even for the simplest non-trivial session type, no coproduct exists.

5.2 Counterexample 2: Two Diamonds (Product Types)

Let $S = \&\{a : \mathbf{end}\} \parallel \&\{b : \mathbf{end}\}$, so $L = \mathcal{L}(S)$ is the 4-element diamond with states $\{(\top_a, \top_b), (\bot, \top_b), (\top_a, \bot), (\bot,\bot)\}$.

Example 5.2. For $L_1 = L_2 = L$ (both 4-element diamonds), the coalesced sum has 8 states, the separated sum has 10, and the linear sum has 7.

The homomorphism enumeration between diamonds is richer. The Reticulate tool finds that there are multiple lattice homomorphisms $L \to L$ (the identity and the constant map $\top \mapsto \top$, all others $\mapsto \bot$, among others). With the product $Y = L \times L$ (a 16-element lattice) as target, the combinatorial explosion of homomorphism pairs (f, g) creates many test cases.

For each construction:

- The **coalesced sum** has cross-factor elements (e.g., $(\top_a, \bot)$ from L_1 and $(\bot, \top_b)$ from L_2) whose meets and joins in the candidate are determined by the shared bottom. The mediating morphism cannot simultaneously satisfy meet-preservation for these cross-factor pairs and the commutativity conditions.
- The **separated sum** isolates the factors, but the fresh top and bottom create additional meet/join constraints. With 10 states and 16 target states, the backtracking search finds zero valid mediating lattice homomorphisms for certain (f, g) pairs.
- The **linear sum** with 7 states stacks the two diamonds. The junction point creates an element that must serve as both the bottom of L_1 and the top of L_2, leading to conflicting constraints when the target separates these roles.

5.3 Counterexample 3: Benchmark Protocols

We also verify the negative result on realistic protocol pairs from the benchmark suite.

Example 5.3. For the Java Iterator protocol $S_1 = \mu X. \&\{hasNext : \oplus\{TRUE : \&\{next : X\}, FALSE : \mathbf{end}\}\}$ (4 states, 2 SCCs) and the File Object protocol $S_2 = \&\{open : \mu X. \&\{read : \oplus\{data : X, eof : \&\{close : \mathbf{end}\}\}\}\}$ (5 states, 4 SCCs), none of the three constructions yields a coproduct. The Reticulate tool reports specific counterexamples:

- **Iterator + File Object**: The coalesced sum has $4 + 5 - 1 + 1 = 9$ states. With SCCs from the recursive Iterator type, the quotient lattice check succeeds (the candidate is a lattice), but the universal property fails when tested against the product $L_1 \times L_2$ (20-state lattice): no valid mediating lattice homomorphism exists.
- **Branch + Selection**: For a branch type $\&\{a : \mathbf{end}, b : \mathbf{end}\}$ and a selection type $\oplus\{a : \mathbf{end}, b : \mathbf{end}\}$, both have 2 states. Despite their structural duality (branch vs. selection), the coproduct analysis fails identically: the fresh top in the coalesced sum violates meet preservation.

5.4 Pattern of Failure

Across all counterexamples, the failure follows a consistent pattern:

1. The "fresh" states (fresh top, fresh bottom, or junction points) are *unconstrained* by the commutativity conditions ($[f, g] \circ \iota_i = f_i$), since they are not in the image of any injection.
2. Any assignment to these fresh states must simultaneously satisfy order-preservation *and* meet/join preservation with respect to the constrained (factor) states.
3. The meet/join relations between factor elements *across* the two copies create constraints that are incompatible with the forced assignments from commutativity.
4. Specifically, elements from L_1 and L_2 that map to the same target element (e.g., both tops mapping to $\top_Y$) must have their meet in C also map to the meet in Y—but in C, their meet is the shared bottom (or fresh bottom), which maps to $\bot_Y$, not to $\top_Y \wedge \top_Y = \top_Y$.

6 Root Cause Analysis

6.1 The Lattice Homomorphism Constraint

The fundamental obstacle is that morphisms in **SessLat** are *lattice homomorphisms*, not merely order-preserving maps. If we relaxed the morphism requirement to monotone maps (forming the category **Pos** of posets), coproducts would exist as disjoint unions with fresh extrema. It is the meet and join preservation requirement that creates the obstruction.

Remark 6.1. In the category of bounded posets with monotone maps, the separated sum *does* satisfy the universal property. The fresh top and bottom are unconstrained, and any order-preserving assignment works. The lattice homomorphism requirement forces the mediating morphism to respect the algebraic structure, which the fresh states cannot accommodate.

6.2 The Free Product in BLat

In the category **BLat** of all bounded lattices (not just session type lattices), coproducts do exist. They are constructed as *free products*: the free bounded lattice generated by the two factors subject to identifying their tops and bottoms. For two 2-element chains, the free product is the

4-element Boolean lattice 2^2—a diamond. For larger factors, the free product grows rapidly: the free product of two 3-element chains has 18 elements, and for general lattices the free product can be infinite [6].

The key difference between the free product and our candidate constructions is that the free product introduces *all necessary elements* to close under meets and joins of cross-factor pairs. In the coalesced sum $L_1 \oplus_\perp L_2$, elements from L_1 and L_2 have meets and joins determined solely by the shared bottom and fresh top. The free product, by contrast, introduces new elements for every cross-factor meet and join that cannot be expressed using existing elements. This closure process is what makes the free product large but ensures the universal property holds.

However, this free product is *not* the state space of any session type in general. Session type lattices have additional structural constraints—reticular form (every non-bottom state must be a pure branch, pure select, or product state), deterministic transitions (each label leads to exactly one state from any given state), and finite state count. The free product may violate any or all of these constraints. **SessLat** is a *non-full* subcategory of **BLat** in the sense that not every bounded lattice is realizable as a session type state space, and the free product witnesses this gap.

Remark 6.2. The "closure gap" between our candidate constructions and the true free product can be quantified. For two 2-element chains, our coalesced sum has 4 states while the free product also has 4 (the Boolean lattice 2^2). However, the lattice structures differ: the coalesced sum has a path graph $\top' \to \top_1, \top_2 \to \perp$ while the free product has the diamond $\top \to a, b \to \perp$. The free product's diamond is realizable as $\mathcal{L}(\&\{a : \mathbf{end}\} \parallel \&\{b : \mathbf{end}\})$, but this is not the coproduct of the factors in **SessLat**—it is their product. The coincidence of the coproduct in **BLat** with the product in **SessLat** for this special case illustrates the fundamental mismatch between the two categories.

6.3 Restricted Subcategories

While general coproducts do not exist, we identify two restricted settings where they may:

1. **Chains.** In the subcategory of **SessLat** consisting of totally ordered lattices (chains), the linear sum $L_1 \diamond L_2$ is well-defined, realizable, and preserves the chain property. However, it still fails the universal property in **SessLat** because the mediating morphism must be a lattice homomorphism in the full category, not just in the subcategory.

2. **Trivial cases.** The 1-element lattice $\mathcal{L}(\mathbf{end})$ acts as an initial object: there is a unique lattice homomorphism from $\mathcal{L}(\mathbf{end})$ to any lattice L (mapping the single element to $\perp_L$). Wait—actually this is wrong. A homomorphism from a 1-element lattice (where $\top = \perp$) to a lattice L with $\top_L \neq \perp_L$ does not exist, because it would have to map $\top \mapsto \top_L$ and $\perp \mapsto \perp_L$ simultaneously. So $\mathcal{L}(\mathbf{end})$ is *not* initial in **SessLat**.

Remark 6.3. The 1-element lattice is neither initial nor terminal in **SessLat**. It is not initial because a homomorphism must preserve $\top$ and $\perp$, and when $\top = \perp$ in the source, the target must also satisfy $\top = \perp$. It is terminal because every lattice L admits a unique homomorphism to any 1-element lattice: map everything to the single element. Wait—this also fails, because a lattice homomorphism must preserve top: $h(\top_L)$ must be $\top_{\mathbf{1}}$, and preserve bottom: $h(\perp_L)$ must be $\perp_{\mathbf{1}}$. Since $\top_{\mathbf{1}} = \perp_{\mathbf{1}}$, this is just the constant map, which does preserve meets and joins. So the 1-element lattice *is* terminal in **SessLat**.

6.4 Asymmetry Between Products and Coproducts

The asymmetry between products and coproducts in **SessLat** is striking. Products exist because the parallel constructor $\parallel$ provides a *canonical* construction with well-behaved projec-

tions. Coproducts would require a dual construction—a canonical "sum" with well-behaved injections—but no such constructor exists in the session type grammar.

This asymmetry reflects a deeper fact about session types: parallel composition ($\|$) is a fundamental operation with clear operational meaning (two concurrent protocol threads), whereas there is no natural session type operation corresponding to "offer a choice between two unrelated protocols." The branch constructor $\&\{m_1 : S_1, \ldots\}$ offers choices, but these choices share a common protocol context and their state spaces merge at common endpoints. A coproduct would require truly disjoint protocol alternatives with a universal factoring property, which the session type framework does not support.

7 Empirical Verification

The negative result is verified mechanically using the Reticulate Python library (version 0.1.0), which provides:

- `coalesced_sum(L1, L2)`, `separated_sum(L1, L2)`, `linear_sum(L1, L2)` — the three candidate constructions.
- `check_coproduct(L1, L2)` — exhaustive analysis trying all three constructions and reporting the best candidate or counterexample.
- `check_coproduct_universal_property` — verifies the universal property by enumerating all lattice homomorphisms from the candidate to test targets and filtering by commutativity.

7.1 Homomorphism Enumeration

A critical component is the exhaustive enumeration of all lattice homomorphisms between small lattices. The algorithm uses backtracking with the following constraints:

1. **Fixed extrema**: $h(\top_S) = \top_T$ and $h(\bot_S) = \bot_T$ (required for lattice homomorphisms of bounded lattices).
2. **Compatibility check**: If source $\top = \bot$ but target $\top \neq \bot$, no homomorphism exists (immediate rejection).
3. **Order pruning**: Each candidate assignment $s \mapsto t$ is checked for order-compatibility with already-assigned states.
4. **Meet/join verification**: After a complete assignment is found, meet and join preservation are checked exhaustively on representative pairs (modulo SCCs).

This enumeration is exponential in the worst case but tractable for the small lattices arising in session types (typically < 15 states).

7.2 Test Coverage

The test suite comprises 55 tests organized into the following categories:

All 55 tests pass, and the full Reticulate test suite (2,670 tests) remains green, confirming no regressions.

7.3 Benchmark Results

We tested coproduct analysis on several benchmark protocol pairs:

Category	Tests	Count
Construction: coalesced sum	State counts, transitions, labels, fresh top	7
Construction: separated sum	State counts, isolation, fresh extrema	6
Construction: linear sum	State counts, join point, symmetry	7
Lattice properties	Lattice checks on candidates	3
Injections	Injectivity, order-preservation	6
Homomorphism enumeration	Identity, end-to-chain, chain-to-end	5
Universal property failure	Coalesced, separated, linear UP checks	5
Main `check_coproduct`	All candidates, flags, negative results	6
Realizability	Reticular form on candidates	4
`is_coproduct` wrapper	Boolean checks	2
Benchmark sweep	Protocol pairs, counterexample reporting	4
Total		**55**

Table 1: Test distribution for the coproduct module.

Protocol L_1	Protocol L_2	States	Coproduct?
$\&\{a : \mathbf{end}\}$ (chain)	$\&\{a : \mathbf{end}\}$ (chain)	$2 + 2$	No
Diamond ($\|\|$)	Diamond ($\|\|$)	$4 + 4$	No
$\&\{a : \mathbf{end}\}$ (chain)	Diamond ($\|\|$)	$2 + 4$	No
$\&\{a : \mathbf{end}, b : \mathbf{end}\}$	$\oplus\{a : \mathbf{end}, b : \mathbf{end}\}$	$2 + 2$	No
Java Iterator	File Object	$4 + 5$	No

Table 2: Coproduct analysis on benchmark protocol pairs. All pairs confirm the negative result.

8 Related Work

Coproducts in lattice theory. In the category **Lat** of all lattices, coproducts (free products) exist by general algebraic arguments [8]. The free product $L_1 * L_2$ is the lattice generated by the disjoint union $L_1 \sqcup L_2$ subject only to the relations holding within each factor. Crucially, free products of lattices can be infinite even for finite factors [6]. Dean [5] showed that the free product of two chains is always finite, a result that partially motivated our investigation of the linear sum construction. The distinction between free products in **Lat** and our constructions is fundamental: free products add *all necessary* meets and joins of cross-factor elements, while our candidates leave cross-factor meets and joins determined by the fresh states (top, bottom, or junction point), which turns out to be insufficient.

Sum types in session types. Some session type systems include a "sum" or "variant" type $S_1 + S_2$ [9, 7], but these are typically syntactic sugar for a branch with two arms, not a categorical coproduct. The operational semantics of such sums involves a runtime choice (the environment selects which branch to follow), not a universal factoring property. In Honda, Vasconcelos, and Kubo [9], the internal choice $\oplus\{l_1 : S_1, l_2 : S_2\}$ has a dual branching structure, but both alternatives share the same channel and session context. Our analysis shows that this shared context is not incidental—it is a structural necessity. A truly "disjoint" sum with a universal property would require the two protocols to have independent state spaces, which the session type framework does not support at the lattice level.

Category theory and session types. Caires and Pfenning [1] established deep connections between session types and linear logic through a Curry-Howard correspondence, where the additive connectives & and $\oplus$ correspond to external and internal choice. Their framework

embeds in a $*$-autonomous category, which has both products and coproducts (the additives). Wadler [10] further developed this correspondence under the name "Propositions as Sessions." Our result shows that when session types are viewed through the lattice lens (the **SessLat** category) rather than the logical lens, the coproduct structure is lost. This is not a contradiction: the categories are different. In the Caires-Pfenning/Wadler framework, the morphisms are process terms (proofs), while in **SessLat** the morphisms are lattice homomorphisms. The lattice perspective captures the *state-space structure* rather than the *proof structure*, and these two views of session types have different categorical properties.

Non-existence results in subcategories. Negative results about categorical limits in subcategories are common in algebra and type theory. For example, the category of finite groups has coproducts (free products) but they are often infinite, leading to their absence in the subcategory of finite groups with finitary morphisms. The category of Noetherian rings is not closed under infinite products. More relevantly, the category of finite distributive lattices has coproducts (given by products of their Birkhoff representations), but these are not generally available in proper subcategories. Our situation is analogous: **BLat** has coproducts, but **SessLat** $\subset$ **BLat** does not, because the realizability constraint excludes the lattices that would serve as coproducts.

Duality and self-duality. In categories arising from linear logic, there is often a duality functor $(-)^{\perp}$ that swaps products and coproducts. In **SessLat**, the duality operation $\overline{S}$ (swapping $\&\{\}$ and $\oplus\{\}$) gives a functor that preserves the lattice up to isomorphism (Step 8 [11]), but it does not turn products into coproducts. The product $\mathcal{L}(S_1 \parallel S_2)$ is symmetric in the sense that $\overline{S_1 \parallel S_2} \cong \overline{S_1} \parallel \overline{S_2}$, so duality preserves the product structure rather than dualizing it into a coproduct. This further confirms the asymmetry: there is no "co-parallel" constructor.

9 Discussion and Future Work

9.1 Summary

We have shown that **SessLat** does not have general coproducts. Three natural candidate constructions—coalesced sum, separated sum, and linear sum—all fail the universal property. The root cause is that the "fresh" states (fresh top, fresh bottom, or junction points) cannot simultaneously satisfy the commutativity conditions and the meet/join preservation requirements of lattice homomorphisms.

9.2 Implications for Protocol Design

Asymmetric categorical structure. **SessLat** has products (from $\parallel$) and a terminal object (the 1-element lattice $\mathcal{L}(\mathbf{end})$), making it a category with finite products. The absence of coproducts means it is *not* bicartesian. This constrains the algebraic operations available for session type composition: we can parallelize protocols but we cannot take their categorical sum.

This asymmetry has practical consequences. In a bicartesian category, one can define both "product protocols" (run S_1 and S_2 in parallel) and "sum protocols" (choose between S_1 and S_2 with a universal factoring law). The absence of sums means that any mechanism for protocol selection (e.g., service discovery, API versioning, protocol negotiation) must be modeled *within* the session type grammar (using branches and selections) rather than as an external categorical operation.

Protocol composition in practice. When a system offers multiple alternative protocols—for example, an API server that supports both REST and GraphQL, or a messaging system that can use either AMQP or MQTT—the "sum" of these protocols cannot be expressed as a categorical coproduct in **SessLat**. Instead, one must model the choice as a branch at the session

type level: $\&\{rest : S_{\text{REST}}, graphql : S_{\text{GraphQL}}\}$. This branch shares the session type context (both alternatives terminate at the same **end** state), which is precisely the structural constraint that coproducts would relax.

Realizability. The realizability constraint (reticular form) plays a secondary role in the failure. Even if we relax realizability and consider all bounded lattices, the coproducts in **BLat** (free products) are typically not finite or not session-type-shaped. The primary obstruction is algebraic, not syntactic.

9.3 Applications and Use Cases

Despite the negative result, the analysis yields practical insights:

Service orchestration. In microservice architectures, services often need to be composed. The product ($\|$) models *concurrent* service invocation: call service A and service B simultaneously. The failed coproduct tells us that *alternative* service invocation (call service A *or* service B) cannot be captured by a universal construction in **SessLat**. This justifies the common pattern of using a dispatcher or router (a branch state) to select between services, rather than attempting a generic "service sum."

Protocol versioning. When a protocol evolves from version V_1 to version V_2, one might wish to define $V_1 + V_2$ as the "versioned protocol" that supports both. The negative result shows that no such construction has a universal property, explaining why protocol versioning typically requires explicit version negotiation (e.g., TLS version handshake, HTTP content negotiation) rather than a generic algebraic mechanism.

Type system design. For language designers building session type systems, the result advises against introducing a "sum type" operator ($S_1 + S_2$) with the expectation that it will behave like a categorical coproduct. Instead, choices between protocols should be modeled using the existing branch/selection constructors, which have well-understood operational semantics even if they lack universal properties.

Formal verification. In formal methods, the absence of coproducts limits the kinds of compositional reasoning available. With products, one can decompose a parallel protocol into independent components and verify each separately. Without coproducts, one cannot similarly decompose a "choice between protocols" into independent verification tasks with a universal assembly principle.

9.4 Open Questions

1. **Pushouts.** Do pushouts exist in **SessLat**? Pushouts generalize coproducts and are needed for amalgamation of protocols sharing a common sub-protocol. A pushout of $L_1 \leftarrow L_0 \rightarrow L_2$ would model "combine L_1 and L_2 while identifying their shared sub-protocol L_0." If pushouts exist for non-trivial spans, this would partially compensate for the absence of coproducts.

2. **Enriched coproducts.** If we weaken the morphisms (e.g., to order-preserving maps, or Galois connections), do coproducts appear? This would define a different category—call it $\mathbf{SessLat}_{\leq}$ for the monotone variant—with potentially richer colimits. Our analysis in Remark 6.1 suggests that the separated sum works in the monotone category.

3. **Equalizers and pullbacks.** Products exist; do equalizers? If so, **SessLat** would have all finite limits (making it finitely complete), even without colimits. A finitely complete category supports kernel pairs, equivalence relations, and quotient-like constructions.

4. **Subcategory coproducts.** In the subcategory of chains (totally ordered session types), does the linear sum become a coproduct? Our initial analysis suggests it still fails in the full **SessLat**, but it may succeed within the chain subcategory if we restrict the morphisms accordingly.

5. **Monoidal structure.** If **SessLat** is not bicartesian, is it at least monoidal? The product from $\|$ gives a symmetric monoidal structure with the 1-element lattice as unit. Is there a separate tensor that differs from the product, perhaps related to sequential composition?

6. **Adjunctions.** Does the forgetful functor $U\colon \mathbf{SessLat} \to \mathbf{BLat}$ have a left adjoint (a "free session type lattice" construction)? If so, the adjunction could provide an indirect route to colimits via the ambient category **BLat**.

9.5 Relationship to the 1000-Steps Programme

Step 164 completes the coproduct analysis within Phase II (Morphism Hierarchy) of the Reticulate programme. The negative result is itself informative: it delineates the boundary of **SessLat**'s categorical structure and motivates the investigation of other categorical properties (equalizers, pullbacks, monoidal structure) in future steps. Together with Step 163 (products), this step provides a complete picture of the binary limit/colimit landscape of **SessLat**:

Categorical property	**Exists?**	**Construction**
Terminal object	Yes	$\mathcal{L}(\mathbf{end})$
Initial object	No	—
Binary products	Yes	$\|$ (parallel)
Binary coproducts	**No**	—
Equalizers	Open	—
Coequalizers	Open	—

The programme continues with the investigation of equalizers, pullbacks, and the monoidal structure of **SessLat**, building toward a complete characterization of its categorical properties.

References

[1] L. Caires and F. Pfenning. Session types as intuitionistic linear propositions. In *CONCUR 2010*, LNCS 6269, pp. 222–236. Springer, 2010.

[2] A. Zua Caldeira. The category SessLat: Products, projections, and the universal property of parallel composition. Reticulate Programme, Step 163, 2026. Preprint.

[3] A. Zua Caldeira. Session types as algebraic reticulates. Reticulate Programme, Step 5, 2026. Preprint.

[4] A. Zua Caldeira. State-space construction for session types. Reticulate Programme, Step 1, 2026. Preprint.

[5] R. Dean. Free lattices generated by partially ordered sets and preserving bounds. *Canadian Journal of Mathematics*, 16:136–148, 1964.

[6] R. Freese, J. Ježek, and J. B. Nation. *Free Lattices*. Mathematical Surveys and Monographs, vol. 42. AMS, 1995.

[7] S. J. Gay and M. J. Hole. Subtyping for session types in the pi calculus. *Acta Informatica*, 42(2–3):191–225, 2005.

[8] G. Grätzer. *Lattice Theory: Foundation*. Birkhäuser/Springer, 2011.

[9] K. Honda, V. T. Vasconcelos, and M. Kubo. Language primitives and type discipline for structured communication-based programming. In *ESOP 1998*, LNCS 1381, pp. 122–138. Springer, 1998.

[10] P. Wadler. Propositions as sessions. In *ICFP 2012*, pp. 273–286. ACM, 2012.

[11] A. Zua Caldeira. Session type duality and state-space isomorphism. Reticulate Programme, Step 8, 2026. Preprint.

DRAFT